

ANN Viva – Questions & Answers (Easy Language)

◆ SECTION 1: Basics of ANN

1. What is an Artificial Neural Network? An ANN is a computer system inspired by the human brain. It consists of connected nodes (neurons) that learn from data and make predictions or decisions.

2. Why are ANNs inspired by the human brain? Because the brain processes information through billions of connected neurons. ANNs mimic this structure — neurons receive input, process it, and pass output to the next layer, just like brain cells.

3. What are the main components of a neuron?

- **Inputs** – data fed into the neuron
- **Weights** – importance given to each input
- **Bias** – a constant added to shift the output
- **Activation function** – decides the final output

4. What is the role of weights and bias in ANN?

- **Weights** control how much each input influences the output.
- **Bias** helps the model shift the activation function, allowing more flexibility. Together, they are adjusted during training to improve accuracy.

5. What is an activation function? It is a mathematical function applied to the neuron's output. It decides whether the neuron should "fire" (activate) or not. Example: ReLU, Sigmoid, Tanh.

6. Why do we need activation functions? Without them, no matter how many layers we have, the network behaves like a simple linear model. Activation functions add **non-linearity**, allowing the network to learn complex patterns.

7. Differentiate between biological neuron and artificial neuron.

Biological Neuron	Artificial Neuron
Found in brain	Found in ANN
Uses electrical signals	Uses numerical values
Dendrites = Inputs	Input values
Synapse = Weights	Weighted connections
Axon = Output	Output value

◆ SECTION 2: Types of Learning

8. What is supervised learning? The model is trained on **labeled data** (input + correct output). It learns to map inputs to outputs. Example: Predicting house prices.

9. What is unsupervised learning? The model is trained on **unlabeled data**. It finds hidden patterns on its own. Example: Customer segmentation (clustering).

10. What is reinforcement learning? The model learns by **trial and error**. It gets rewards for correct actions and penalties for wrong ones. Example: Game-playing AI (like Chess or Go).

◆ SECTION 3: Network Architecture

11. What is a single-layer perceptron? The simplest ANN with only **one input layer and one output layer** (no hidden layers). It can only solve linearly separable problems (like AND, OR gate).

12. What is a multilayer perceptron (MLP)? An ANN with **one or more hidden layers** between input and output. It can solve complex, non-linear problems like image recognition.

13. What are input, hidden, and output layers?

- **Input layer** – receives raw data
- **Hidden layers** – process and transform the data
- **Output layer** – gives the final prediction/result

14. Why are hidden layers important? They allow the network to learn complex patterns. Each hidden layer extracts higher-level features from the previous layer's output.

15. What is deep learning? Deep learning uses **neural networks with many hidden layers** (deep networks). It is used for complex tasks like image recognition, language translation, and speech recognition.

16. Differentiate between shallow and deep neural networks.

Shallow Network	Deep Network
1-2 hidden layers	Many hidden layers
Less powerful	More powerful
Faster training	Slower training
Simpler problems	Complex problems

17. What is the purpose of neurons in hidden layers? They learn to detect **patterns and features** in data. For example, in an image, early layers detect edges, middle layers detect shapes, and deeper layers detect objects.

18. What is a feedforward neural network? Data flows in **one direction only** — from input → hidden layers → output. There is no loop or feedback. Most basic ANNs are feedforward.

19. What is recurrent feedback in neural networks? In recurrent networks (RNN), the output of a neuron can be fed **back as input** in the next step. This helps the network remember past information (used in time series, text, speech).

20. What are the limitations of a single-layer perceptron?

- Cannot solve non-linearly separable problems (like XOR)
- No hidden layers means very limited learning capacity
- Cannot learn complex patterns

◆ SECTION 4: Activation Functions

21. Explain the sigmoid activation function. Formula: $\sigma(x) = 1 / (1 + e^{(-x)})$ Output is always between **0 and 1**. Useful for binary classification. Makes output look like a probability.

22. Advantages and disadvantages of sigmoid.

-  Output between 0 and 1 (good for probability)
-  Smooth gradient
-  Vanishing gradient problem

-  Output is not zero-centered

23. What is the ReLU activation function? Formula: $f(x) = \max(0, x)$ If input is negative → output is 0. If positive → output is the same value. Simple and fast.

24. Why is ReLU widely used?

- Fast to compute
- Avoids vanishing gradient problem
- Works well in deep networks
- Most common activation function in practice

25. What is the tanh activation function? Formula: $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$ Output is between **-1 and 1**. Zero-centered, so better than sigmoid in many cases.

26. Differentiate between sigmoid and tanh.

Sigmoid	Tanh
Output: 0 to 1	Output: -1 to 1
Not zero-centered	Zero-centered
Slower convergence	Faster convergence

27. What is softmax activation? Converts a list of numbers into **probabilities that add up to 1**. Used in the output layer for **multi-class classification**.

28. In which applications is softmax used?

- Handwritten digit recognition (0-9)
- Image classification (cat/dog/bird)
- Any multi-class prediction problem

29. What is the vanishing gradient problem? During backpropagation, gradients get **very small** as they move backward through many layers. This makes the weights barely update, so the network stops learning — especially in deep networks.

30. Which activation functions help reduce vanishing gradients?

- **ReLU** (most effective)
 - **Leaky ReLU**
 - **ELU** These functions don't squash gradients like sigmoid/tanh do.
-

◆ SECTION 5: Training Concepts

31. What is training in ANN? The process of **adjusting weights and biases** of the network using training data so that it can make accurate predictions.

32. What is an epoch? One complete pass through the **entire training dataset**. If you have 1000 samples and run 10 epochs, the model sees each sample 10 times.

33. What is batch size? The number of training samples used in **one update** of the model's weights. Example: batch size = 32 means weights are updated after every 32 samples.

34. What is iteration in ANN training? One weight update step. If dataset = 1000 samples, batch size = 100, then 1 epoch = **10 iterations**.

35. What is forward propagation? The input data moves **forward** through the network — from input → hidden layers → output layer — to generate a prediction.

36. What is backpropagation? After forward propagation, the error is calculated. Backpropagation **sends this error backward** through the network, computing gradients and updating weights to reduce the error.

37. Why is backpropagation important? It is the **core learning algorithm** of ANN. Without it, we can't adjust weights to minimize error. It makes neural networks actually learn.

38. Explain gradient descent. An optimization algorithm that **minimizes the loss function** by updating weights in the direction of the steepest decrease (negative gradient).
$$\text{New Weight} = \text{Old Weight} - \text{Learning Rate} \times \text{Gradient}$$

39. What is learning rate? A small number (e.g., 0.01) that controls **how big each weight update** is during training. It's the speed of learning.

40. What happens if the learning rate is too high or too low?

- **Too high** → Model overshoots the minimum, may never converge (unstable)
 - **Too low** → Model learns very slowly, takes too long to converge
-

◆ SECTION 6: Loss Functions & Optimization

41. What is a loss function? A function that measures **how wrong the model's prediction** is for a single training example. Lower loss = better model.

42. Differentiate between loss function and cost function.

- **Loss function** → error for one sample
- **Cost function** → average error over all samples (the whole dataset)

43. What is Mean Squared Error (MSE)? $MSE = (1/n) \times \sum(\text{actual} - \text{predicted})^2$ Used for **regression problems**. Penalizes large errors more due to squaring.

44. What is cross-entropy loss? Used for **classification problems**. Measures how different the predicted probability distribution is from the actual labels. Lower = better.

45. What is optimization in ANN? The process of **adjusting weights to minimize the cost function**. It decides how weights are updated during training.

46. Name different optimization algorithms.

- Gradient Descent (GD)
- Stochastic Gradient Descent (SGD)
- Mini-batch Gradient Descent
- Adam
- RMSprop
- Adagrad

47. What is stochastic gradient descent? Updates weights after **each single training sample** (instead of the whole dataset). Faster but noisier updates. Good for large datasets.

48. Differentiate between batch GD and SGD.

Batch GD	Stochastic GD
Uses full dataset per update	Uses 1 sample per update
Slow but stable	Fast but noisy
More memory needed	Less memory needed

49. What is overfitting? Model **memorizes** training data too well but fails on new data. Training accuracy is high, test accuracy is low. Like memorizing answers without understanding.

50. What is underfitting? Model is **too simple** to learn the data. Both training and test accuracy are low. It hasn't learned enough.

◆ SECTION 7: Data Preprocessing

51. Why is data preprocessing important? Raw data is messy, unscaled, or missing values. Preprocessing makes it **clean and ready** for the ANN so it can learn better and faster.

52. What is normalization? Scaling data to a **standard range** (like 0 to 1 or -1 to 1) so no feature dominates others. Formula:
$$\frac{(x - \min)}{(\max - \min)}$$

53. What is feature scaling? Adjusting the **range of features** so they are on a similar scale. Prevents features with large values from overshadowing smaller ones during training.

54. Why do we split data into training and testing datasets?

- **Training data** – used to train the model
- **Test data** – used to evaluate how well the model performs on **unseen data** This helps us know if the model generalizes well.

55. What is validation data? A separate portion of data used **during training** to tune hyperparameters (like learning rate, number of layers). It's different from test data.

56. What is one-hot encoding? Converting **categorical labels into binary vectors**. Example: [Dog, Cat, Bird] → Dog=[1,0,0], Cat=[0,1,0], Bird=[0,0,1]. Used so ANN can understand categories as numbers.

57. What type of data is suitable for ANN?

- Numerical data (after scaling)
- Categorical data (after encoding)
- Images (as pixel values)
- Text (after vectorization)
- Audio (as spectrograms)

58. What are categorical and numerical features?

- **Categorical** – discrete labels: color (red, blue), gender (male, female)
- **Numerical** – numbers: age, salary, temperature

59. Why is missing data problematic? ANN cannot process missing values — it causes errors or produces wrong predictions. We must handle them by filling (imputation) or removing the rows.

60. How can ANN handle noisy data?

- Data cleaning before training
 - Dropout regularization
 - Large training datasets
 - Averaging multiple predictions
-

◆ SECTION 8: Python Libraries & Implementation

61. Which libraries are used for ANN in Python? TensorFlow, Keras, PyTorch, Scikit-learn, Theano

62. What is TensorFlow? An open-source **deep learning framework by Google**. Used for building, training, and deploying neural networks. Works on CPU and GPU.

63. What is Keras? A **high-level API** built on top of TensorFlow. It makes building ANNs very easy with just a few lines of code. Beginner-friendly.

64. Differentiate between TensorFlow and Keras.

TensorFlow	Keras
Low-level, more control	High-level, easier to use
More complex	Simpler syntax
Built by Google	Built on top of TF

65. What is PyTorch? An open-source deep learning library by **Facebook/Meta**. Popular in research due to its flexibility and dynamic computation graphs.

66. How do you create a neural network model in Keras?

```
python

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(64, activation='relu', input_shape=(10,)))
model.add(Dense(1, activation='sigmoid'))
```

67. What is the purpose of model compilation? It configures the model for training — specifying the **optimizer, loss function, and metrics**.

68. What parameters are specified during model compilation?

```
python

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

- `optimizer` - how weights are updated

- `loss` - how error is measured
- `metrics` - what to monitor (accuracy)

69. What is model fitting? Training the model on data:

```
python

model.fit(X_train, y_train, epochs=10, batch_size=32)
```

The model learns by adjusting weights over multiple epochs.

70. Explain the use of `Sequential()` model in Keras. It creates a **linear stack of layers** where data flows from one layer to the next in sequence. Easy for building simple feedforward networks.

71. What is the role of Dense layer? A fully connected layer where **every neuron is connected to every neuron in the previous layer**. The most basic layer in Keras.

72. How do you evaluate a trained model?

```
python

loss, accuracy = model.evaluate(X_test, y_test)
```

It checks model performance on unseen test data.

73. What is the use of `predict()` function?

```
python

predictions = model.predict(X_new)
```

Used to get the **model's output/prediction** on new, unseen data.

74. What is accuracy in ANN? $\text{Accuracy} = (\text{Correct Predictions} / \text{Total Predictions}) \times 100$ It tells you how often the model predicts correctly. Higher is better.

75. How can you improve model accuracy?

- Add more training data
 - Increase/decrease hidden layers or neurons
 - Use better activation functions
 - Tune learning rate
 - Apply dropout to prevent overfitting
 - Use batch normalization
-

◆ SECTION 9: Mathematics Behind ANN

76. Why is differentiation used in backpropagation? To find how much each weight contributes to the error. The derivative (gradient) tells us in which direction and how much to update the weight.

77. What is chain rule in ANN? A calculus rule used to compute gradients through multiple layers. $(dL/dw = dL/dout \times dout/dnet \times dnet/dw)$ It chains the derivatives of each layer together during backpropagation.

78. Explain weight update rule. $(New\ Weight = Old\ Weight - (Learning\ Rate \times Gradient))$
The weight is adjusted in the direction that reduces the loss.

79. What is the role of derivatives in ANN learning? Derivatives measure how much the loss changes when a weight changes. This guides the optimization algorithm to update weights correctly.

80. Why should activation functions be differentiable? Because backpropagation requires computing the **gradient of the activation function**. If it's not differentiable, we can't compute gradients and weights can't be updated.

◆ SECTION 10: Practical Questions

81. Explain the ANN experiment you performed. We built a neural network model using Keras to classify/predict [your target]. We preprocessed the data, built the model with input-hidden-output layers, compiled it, trained it, and evaluated its performance.

82. What dataset did you use in your practical? (*Mention your dataset — e.g., Iris dataset, MNIST, Pima Diabetes, etc.*)

83. What preprocessing steps were applied?

- Handled missing values
- Applied normalization/scaling
- Encoded categorical variables (one-hot encoding)
- Split into train/test sets

84. Which activation function did you use and why?

- **Hidden layers:** ReLU — fast, avoids vanishing gradient
- **Output layer:** Sigmoid (binary) or Softmax (multi-class)

85. How many hidden layers did your model contain? (*Mention your actual model — e.g., 2 hidden layers with 64 and 32 neurons*)

86. Which optimizer did you use? Adam optimizer — it combines the benefits of SGD with momentum and is the most commonly used adaptive optimizer.

87. What learning rate was selected? Typically **0.001** (default for Adam). Can be tuned for better performance.

88. How did you measure performance? Using **accuracy** on the test set and **loss curves** on training/validation data.

89. What accuracy did you achieve? (*Mention your actual result — e.g., "We achieved 92% accuracy on the test set"*)

90. What challenges did you face during implementation?

- Handling missing/noisy data
 - Choosing the right number of layers and neurons
 - Preventing overfitting
 - Selecting appropriate learning rate
-

◆ SECTION 11: Applications of ANN

91. Mention real-life applications of ANN. Image recognition, speech recognition, medical diagnosis, fraud detection, recommendation systems, autonomous vehicles, weather prediction, language translation.

92. How is ANN used in image recognition? ANN (especially CNNs) learn to detect edges, shapes, and objects in images by training on labeled image datasets. Used in facial recognition, self-driving cars.

93. How is ANN useful in medical diagnosis? ANN can detect diseases from X-rays, MRIs, or patient data. Example: detecting cancer cells, predicting diabetes, reading ECG reports.

94. What is the role of ANN in speech recognition? ANN processes audio signals and maps them to text. Used in Siri, Google Assistant, Alexa.

95. How is ANN used in recommendation systems? ANN learns user preferences from past behavior and suggests relevant content. Used in Netflix, YouTube, Amazon recommendations.

96. Can ANN be used for weather prediction? Yes! ANNs can analyze temperature, pressure, humidity patterns from historical data to predict future weather conditions.

97. How is ANN applied in autonomous vehicles? ANN processes camera/sensor input in real-time to detect roads, traffic signs, pedestrians, and make driving decisions.

◆ SECTION 12: Advantages, Disadvantages & Comparison

98. What are the advantages of ANN?

- Can learn complex patterns
- Works with all types of data
- Improves with more data
- Can handle noisy data
- Generalizes well once trained

99. What are the disadvantages of ANN?

- Requires a lot of data
- Computationally expensive (needs powerful hardware)
- Black box — hard to interpret
- Prone to overfitting
- Many hyperparameters to tune

100. Compare ANN with traditional programming approaches.

Traditional Programming	ANN
Rules are manually written	Rules are learned from data
Can't handle complex patterns	Handles complex patterns well
Needs expert knowledge	Learns from examples
Fast and explainable	Slow to train, black box
Fixed behavior	Adaptive behavior

Best of luck for your viva! 🍀